LINEAMIENTOS PARA EL DESARROLLO SEGURO DE SOFTWARE

Sistema Dysto-AI — Marketplace con análisis de imágenes mediante IA

ÍNDICE:

1. INTRODUCCIÓN..... 3
2. CONSIDERACIONES PARA SEGURIDAD EN EL DESARROLLO.....4
2.1 Implementación de librerías y frameworks de seguridad.....4
2.2 Consultas seguras de base de datos.....4
2.3 Validación de datos.....5
2.4 Implementación de mecanismos de autenticación.....5
2.5 Manejo de sesiones.....6
2.6 Calidad de código.....6
2.7 Creación de base de datos y datos iniciales.....6
2.8 Manejo de errores y excepciones.....7
3. TECNOLOGÍAS DE PREFERENCIA.....8
3.1 Lenguajes de programación.....8
3.1 Almacenamiento de datos relacionales.....8
3.2 Almacenamiento de datos no relacionales.....8
3.3 Registro de eventos (Logging).....9
4. USO DE TECNOLOGÍAS.....10
4.1 Lenguaje (Frameworks, Patrones).....10
4.2 Almacenamiento (Uso de cluster y tipos de consultas).....10
4.3 Tareas externas (Fuera de línea o programadas).....10
5. USO DE REPOSITORIOS DE INTEGRACIÓN.....12
5.1 Capacidad de almacenamiento.....12
5.2 Uso de ramas.....12
5.3 Reglas de commit.....12
6. DESARROLLO Y CONSUMO DE APIs.....13
6.1 Integración de servicios (REST).....13
6.2 Autenticación de servicios.....13
6.3 Interoperabilidad (especificación de rutas).....13
6.4 Uso del idioma en el código.....13
7. ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE.....15
7.1 Plan de pruebas.....15
7.2 Diseño de pruebas.....15
7.3 Reporte de resultados.....15

1. INTRODUCCIÓN

El presente documento establece los lineamientos técnicos, organizacionales y de seguridad que deberán seguirse durante el desarrollo del sistema **Dysto-AI**, con el objetivo de garantizar la calidad del software, la protección de los datos de los usuarios y la operación segura del sistema.

Dysto-AI es un sistema tipo marketplace en el cual los productos obtienen sus datos automáticamente mediante análisis de imágenes utilizando inteligencia artificial. El sistema permite publicar artículos, gestionar inventario y realizar compras simuladas.

Esta guía define la arquitectura tecnológica, prácticas de desarrollo seguro, políticas de control, procedimientos de prueba y lineamientos de operación del sistema.

2. CONSIDERACIONES PARA SEGURIDAD EN EL DESARROLLO

La seguridad debe integrarse como un componente estructural del ciclo de vida del software y no como un elemento posterior o correctivo. Todo sistema desarrollado bajo estos lineamientos deberá diseñarse, implementarse y mantenerse bajo principios de prevención, control, trazabilidad y minimización de riesgos.

2.1 Implementación de librerías y frameworks de seguridad

El sistema deberá utilizar las capacidades de seguridad integradas en las tecnologías seleccionadas para el desarrollo de la aplicación, tales como mecanismos de autenticación, validación de datos y control de acceso.

Las funcionalidades de seguridad proporcionadas por los componentes tecnológicos deberán utilizarse conforme a sus configuraciones recomendadas, evitando implementaciones manuales cuando exista una solución integrada disponible.

Asimismo, el sistema podrá apoyarse en componentes o servicios externos que gestionen funciones relevantes para la operación o seguridad del sistema. Estos componentes deberán utilizarse conforme a sus configuraciones estándar y buenas prácticas de integración.

(MENCIONAR DE FORMA UTIL COMO SE COMPROBARIA A TRAVÉS DE LOS ARCHIVOS DE CONFIGURACION O ALGO ASI) MENCIONAR LAS LIBRERIAS DE SEGURIDAD DE USO COMUN EN EL PROYECTO

2.2 Consultas seguras de base de datos

El acceso a la información persistente deberá realizarse mediante mecanismos que separen la lógica de negocio del acceso directo a datos, evitando manipulación manual de instrucciones de consulta.

Se deberán aplicar los siguientes principios:

- Uso de mecanismos de abstracción o acceso controlado a datos

- Prohibición de construcción dinámica de consultas con datos no sanitizados
- Validación previa de toda entrada que impacte almacenamiento

Toda interacción con la base de datos deberá ser auditable y trazable.

FALTAN EJEMPLOS Y UNA MEJOR EXPLICACION SOBRE EL USO DE ORMs y PREPARED STATEMANTS

2.3 Validación de datos

Todo dato recibido por el sistema deberá ser considerado no confiable hasta que sea validado explícitamente.

La validación deberá implementarse en múltiples niveles:

- Interfaz de usuario
- Capa de servicios
- Capa de persistencia

Los mecanismos de validación deberán verificar:

- Formato
- Longitud

Cualquier dato que no cumpla criterios definidos deberá ser rechazado o normalizado antes de su procesamiento.

PONER UN EJEMPLO DE REGEX UNO DE BASE DE DATOS Y UNO DE FRONT

2.4 Implementación de mecanismos de autenticación

Todo sistema deberá implementar un mecanismo formal de identificación de usuarios o entidades que interactúan con la aplicación.

La autenticación deberá garantizar:

- Verificación de identidad
- Protección de credenciales
- Control de acceso basado en privilegios

- Capacidad de revocación de acceso
- Ciclo de vida de credenciales

Se permitirá la integración con proveedores externos de identidad cuando el modelo de seguridad del sistema lo requiera.

2.5 Manejo de sesiones

Toda sesión autenticada deberá ser gestionada mediante mecanismos controlados que permitan mantener, validar y finalizar la interacción del usuario de forma segura.

El sistema deberá contemplar:

- Expiración controlada
- Invalidación explícita
- Renovación opcional de sesión

2.6 Calidad de código

La calidad del código es un factor determinante en la seguridad del sistema. El software deberá desarrollarse bajo principios de claridad, mantenibilidad y trazabilidad.

Todo proyecto deberá establecer:

- Revisión manual de código antes de integración
- Uso consistente de convenciones de nomenclatura
- Utilización de palabras clave funcionales para identificar responsabilidades del código

2.7 Creación de base de datos y datos iniciales

Toda estructura de almacenamiento deberá ser creada mediante procedimientos controlados, versionados y reproducibles.

El sistema deberá permitir:

- Definición estructurada del modelo de datos
- Despliegue controlado de esquemas
- Inicialización consistente del entorno

Los datos iniciales deberán limitarse exclusivamente a los necesarios para la operación básica del sistema.

2.8 Manejo de errores y excepciones

Todo error deberá ser tratado como un evento controlado del sistema.

La aplicación deberá:

- Registrar internamente los errores
- Evitar exponer detalles técnicos al usuario final
- Clasificar fallos según tipo e impacto

La presentación de errores deberá ser comprensible, controlada y coherente con la política de comunicación del sistema.

3. TECNOLOGÍAS DE PREFERENCIA

La selección tecnológica deberá responder a criterios de estabilidad, seguridad, mantenibilidad y compatibilidad con los objetivos del sistema como lo son.

- **Backend:** Python 3.12 con el framework Django 5.0 y Django REST Framework 3.14. Se elige por su alta compatibilidad con el ecosistema de librerías de Inteligencia Artificial, procesamiento de datos y machine learning.
- **Frontend:** JavaScript (ES2023) utilizando el framework Next.js (v14) basado en React 18. Se elige por su capacidad de renderizado del lado del servidor (SSR) y generación estática (SSG), lo cual optimiza drásticamente los tiempos de carga y el posicionamiento en buscadores (SEO) de las publicaciones, manteniendo la alta interactividad de las interfaces dinámicas mediante el Virtual DOM.
- **Inteligencia Artificial:** Modelo multimodal Gemini (a través de su API) para el procesamiento avanzado de imágenes, permitiendo el reconocimiento preciso de objetos y la extracción automática de metadatos variables (como color, material, capacidad y categoría) directamente desde las fotografías de los productos.

3.1 Lenguajes de programación

Los lenguajes utilizados deberán ser apropiados para el tipo de aplicación desarrollada y contar con soporte activo, herramientas de desarrollo maduras y mecanismos de seguridad documentados.

La elección del lenguaje deberá justificarse por:

- Capacidad de mantenimiento a largo plazo
- Ecosistema de herramientas
- Disponibilidad de talento técnico
- Integración con otros componentes del sistema

3.2 Almacenamiento de datos relacionales

El sistema deberá utilizar almacenamiento estructurado que permita integridad, consistencia y control transaccional de la información.

La solución adoptada deberá garantizar:

- Integridad referencial
- Control de concurrencia
- Mecanismos de respaldo
- Capacidad de auditoría

3.3 Almacenamiento de datos no relacionales

No se contempla el uso de almacenamiento no relacional dentro del alcance actual del sistema.

Su adopción futura deberá evaluarse en función de necesidades específicas de escalabilidad o naturaleza de datos.

3.4 Registro de eventos (Logging)

Todo sistema deberá implementar mecanismos de registro que permitan reconstruir la actividad operativa y detectar comportamientos anómalos.

El registro deberá contemplar:

- Eventos del sistema
- Errores de ejecución
- Acciones relevantes de usuarios
- Eventos de seguridad

Los registros serán almacenados dentro de la infraestructura del sistema mediante mecanismos controlados y con acceso restringido.

La conservación de los registros se mantendrá durante el tiempo necesario para fines operativos y de análisis del sistema.

4. USO DE TECNOLOGÍAS

4.1 Lenguaje (Frameworks, Patrones)

La arquitectura del sistema deberá organizarse mediante componentes desacoplados, responsabilidades definidas y separación clara entre capas funcionales.

El diseño deberá favorecer:

- Modularidad
- Reutilización
- Mantenibilidad
- Escalabilidad

La organización del código seguirá principios de separación de responsabilidades entre componentes del sistema.

4.2 Almacenamiento (Uso de cluster y tipos de consultas)

El sistema estructurará el acceso a datos de forma eficiente, consistente y controlada mediante mecanismos provistos por el motor de almacenamiento utilizado.

La arquitectura de almacenamiento deberá permitir:

- Acceso concurrente controlado
- Consistencia de datos
- Capacidad de crecimiento

4.3 Tareas externas (Fuera de línea o programadas)

El sistema podrá ejecutar procesos fuera del flujo principal de interacción del usuario cuando la naturaleza del procesamiento lo requiera.

Estos procesos podrán incluir ejecución asíncrona de operaciones y tareas programadas del sistema.

Las tareas externas deberán:

- ejecutarse de forma controlada
- registrarse para efectos de auditoría

4.4 Estándares de Codificación y Nomenclatura

Para garantizar que el código fuente del sistema Dysto-AI sea legible, uniforme y fácilmente transferible entre cualquier miembro del equipo, es estrictamente obligatorio apegarse a las siguientes convenciones de nomenclatura en todos los entornos (Frontend en Next.js y Backend en Django):

- Clases y Modelos (Python/TypeScript): Se debe utilizar la notación UpperCamelCase (también conocida como PascalCase). Cada palabra debe iniciar con mayúscula y no se deben usar espacios ni guiones.
    - Ejemplo: class UsuarioController:, class ProductoModel:
- Variables, Funciones y Métodos: Se debe emplear la notación lowerCamelCase. La primera palabra inicia en minúscula y las siguientes palabras inician con mayúscula, sin usar tildes ni caracteres especiales.
    - Ejemplo: function calcularTotalCompra(), let informacionUsuario;
- Constantes: Todas las variables globales inmutables o constantes de configuración deben escribirse completamente en mayúsculas, separando las palabras con guiones bajos (UPPER_SNAKE_CASE).
    - Ejemplo: MAX_INTENTOS = 3, const API_BASE_URL = "...";
- Base de Datos (PostgreSQL): Para mantener la compatibilidad con el ORM de Django y los estándares de SQL, los nombres de las tablas y columnas deben escribirse en minúsculas y separarse con guiones bajos (snake_case).
    - Ejemplo: fecha_creacion, id_producto_foraneo.

5. USO DE REPOSITORIOS DE INTEGRACIÓN

La administración del código fuente deberá realizarse exclusivamente mediante repositorios centralizados que permitan trazabilidad completa del historial de cambios.

5.1 Capacidad de almacenamiento

El repositorio oficial deberá almacenar todo artefacto necesario para reproducir el sistema, incluyendo código fuente, configuración y documentación técnica.

5.2 Uso de ramas

El proyecto deberá implementar una estrategia de ramificación que separe claramente:

- Código estable
- Código en integración
- Trabajo en desarrollo

Las ramas protegidas no deberán modificarse sin proceso formal de revisión.

5.3 Reglas de commit

Todo cambio deberá registrarse con mensajes descriptivos que permitan identificar:

- Propósito del cambio
- Alcance funcional
- Naturaleza de la modificación

Es de carácter obligatorio utilizar una convención profesional (como Conventional Commits) que incluya un prefijo específico, seguido de una descripción clara de la modificación. Se deben emplear las siguientes etiquetas estandarizadas:

- [FEAT]: Para la incorporación de nuevas funcionalidades (ej. [FEAT] Creación del login).

- [FIX]: Para la corrección de errores dentro del código (ej. [FIX] Error en validación de correo).

6. DESARROLLO Y CONSUMO DE APIs

6.1 Integración de servicios (REST)

La comunicación entre componentes deberá realizarse mediante interfaces definidas, documentadas y versionadas.

Toda interfaz deberá especificar:

- Métodos permitidos
- Formato de intercambio de datos
- Estructura de respuesta
- Manejo de errores

Ejemplo de Endpoint correcto:

- GET /api/v1/products/{id}

Ejemplo de Endpoint incorrecto:

- POST /obtenerDatosDelProducto?id=15

6.2 Autenticación de servicios

Toda comunicación entre componentes del sistema deberá validar la identidad del solicitante mediante mecanismos de autenticación basados en credenciales verificables.

Las solicitudes que no presenten credenciales válidas o verificables no serán procesadas por el sistema.

Ejemplo de credenciales válidas:

- GET /api/v1/orders
- Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5...

Ejemplo de credenciales inválidas:

- GET /api/v1/orders

- Sin token

6.3 Interoperabilidad (especificación de rutas)

Las rutas de acceso a servicios deberán definirse mediante convenciones uniformes y consistentes que faciliten la comprensión, el mantenimiento y la evolución del sistema.

La estructura de rutas deberá ser predecible, semántica y coherente en todo el sistema.

Ejemplo de ruta correcto:

- PUT /api/v1/products/{id}

Ejemplo de ruta incorrecto:

- /prueba/Borrar-Producto-Ahora

6.4 Uso del idioma en el código

El código fuente deberá emplear nomenclatura técnica consistente en un único idioma para estructuras de programación, incluyendo variables, funciones, clases y componentes.

Los identificadores técnicos se definirán en idioma inglés para mantener compatibilidad con estándares de desarrollo.

Los mensajes orientados al usuario final podrán presentarse en el idioma definido para la interfaz del sistema.

Ejemplo de idioma correcto:

- ```
  function calculateTotalPrice(items) {
  	return items.reduce((sum, item) => sum + item.price, 0);
  }
  ```

Ejemplo de idioma incorrecto:
  ```

- function calcularTotalPrice(listaDeItems) {
  return listaDeItems.reduce((suma, item) => suma + item.price, 0);
  }

7. ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE

7.1 Plan de pruebas

El proyecto deberá contar con un plan de pruebas documentado que establezca la estrategia general para verificar el cumplimiento funcional del sistema.

El plan de pruebas definirá los componentes, módulos o funcionalidades que deberán ser evaluados, así como los criterios de validación aplicables a cada uno.

Para cada elemento evaluado se definirán casos de prueba asociados que permitan verificar su comportamiento esperado. Dichas pruebas se ejecutarán y sus resultados se registrarán como parte del seguimiento de calidad.

7.2 Diseño de pruebas

Las pruebas deberán diseñarse a partir de los comportamientos esperados del sistema, sus reglas operativas y sus requisitos funcionales.

Cada caso de prueba describirá las condiciones de ejecución, el resultado esperado y el criterio de validación correspondiente.

El diseño de pruebas se integrará dentro del plan de pruebas del proyecto.

7.3 Reporte de resultados

Toda ejecución de pruebas deberá registrarse para permitir seguimiento y control del estado funcional del sistema.

Los resultados de cada prueba se documentarán indicando la ejecución realizada y el estado obtenido.

El registro de resultados se integrará dentro del documento del plan de pruebas del proyecto.